Requirements for Multiagent Interaction Protocols

Munindar P. Singh[2] Amit K. Chopra[1] Samuel H. Christie V[2]
(with contributions from Matteo Baldoni[3] and Omkar Joshi[2])

[1]Lancaster University

[2]North Carolina State University

[3]University of Torino

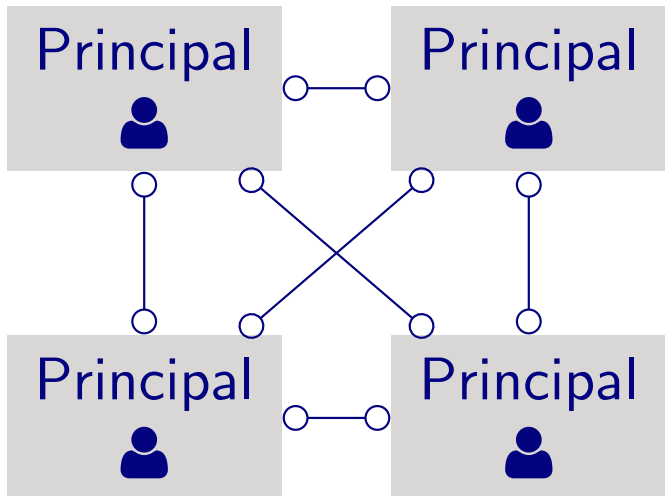
Papers

Code

W3C TPAC 2026

Before Computing: Decentralization was Natural

Long-lived engagements between autonomous principals

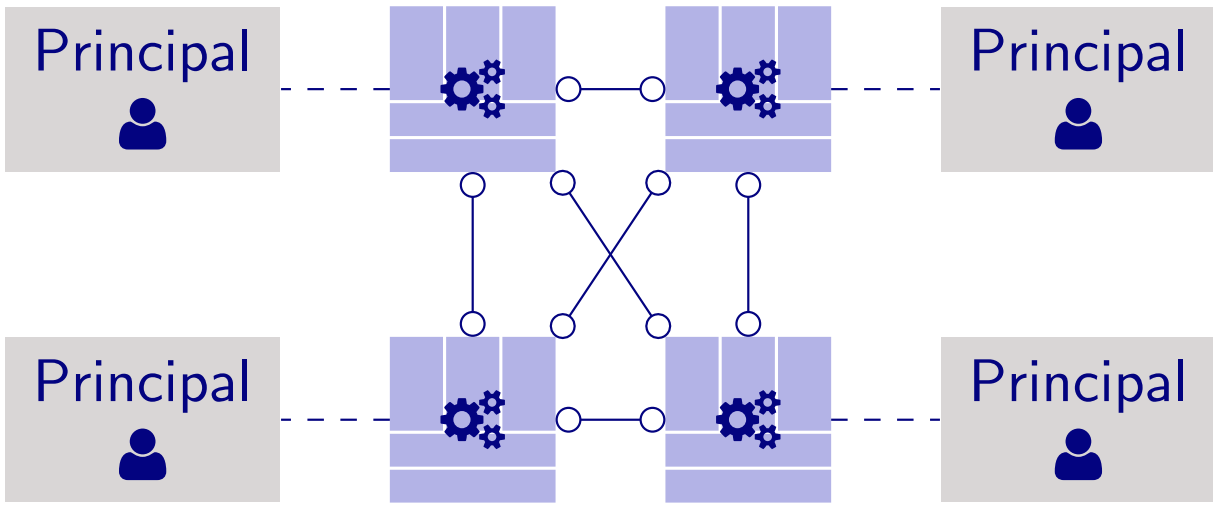


- In business, health, finance, ...
- Conceptually decentralized

Multiagent Systems: Agents Help Principals

Realize a decentralized, loosely coupled system to promote flexibility

Heterogeneous agents encode decision making of their respective principals



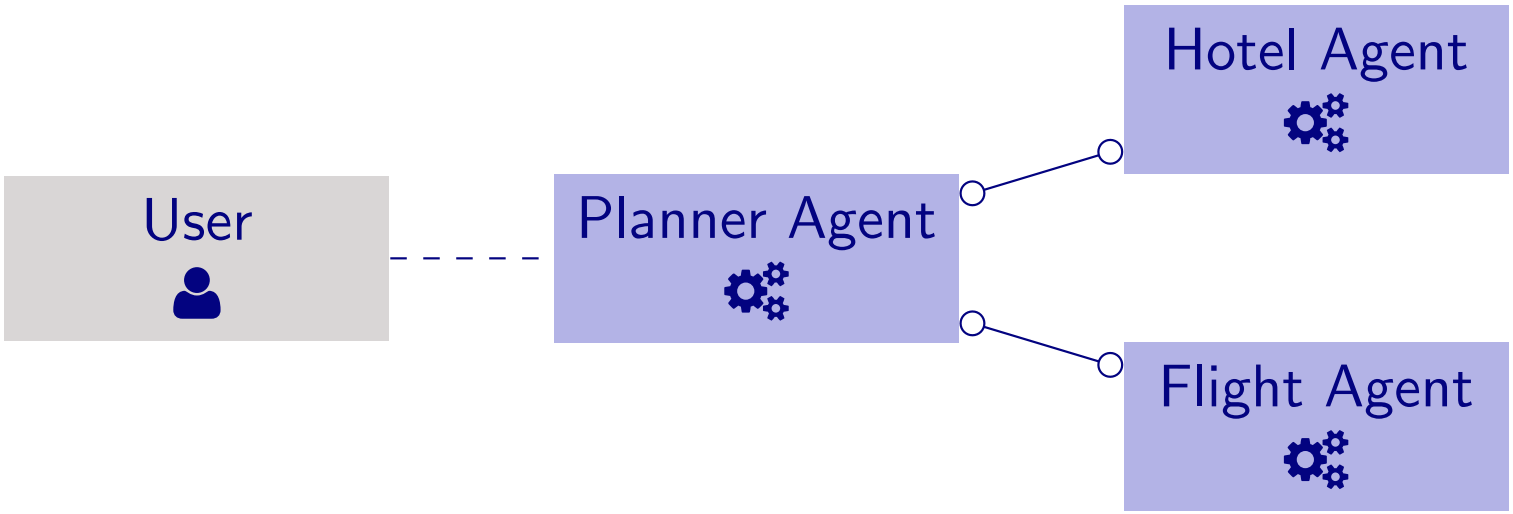
What we need: Protocols at two levels

Operational (Regulative) Constraints on the ordering and occurrence of interactions

Constitutive The meaning of an interaction on the social tier

Agentic AI Today: Multiagent Lite

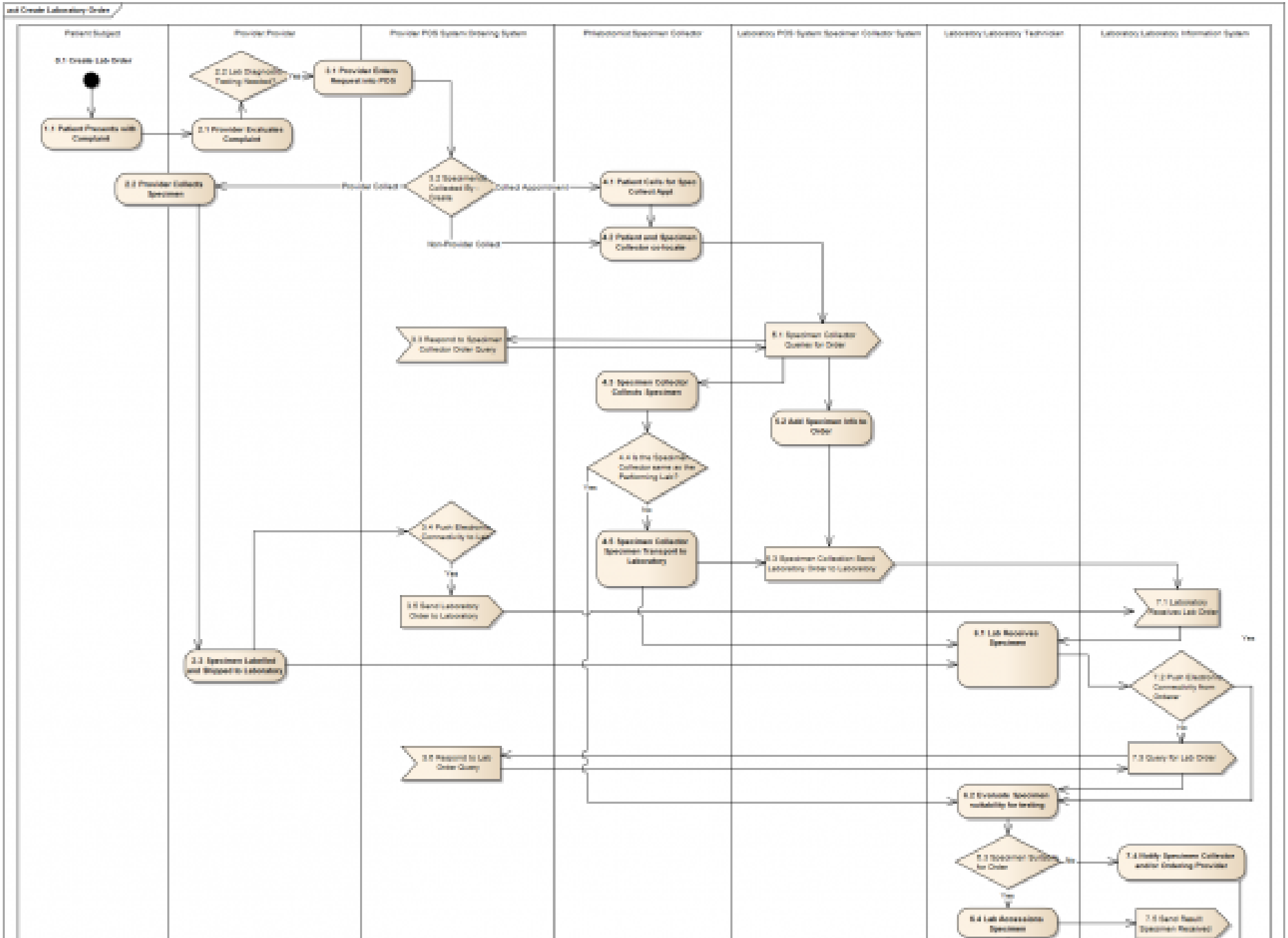
Flexible, generative AI-powered agents that make real-world decisions



Inflexible coordination defeats flexibility

- Workflows and task graphs: Traditional scripting languages
- No meaning
- Inflexible operationalization

Workflows (This is from HL7): Don't try to read!

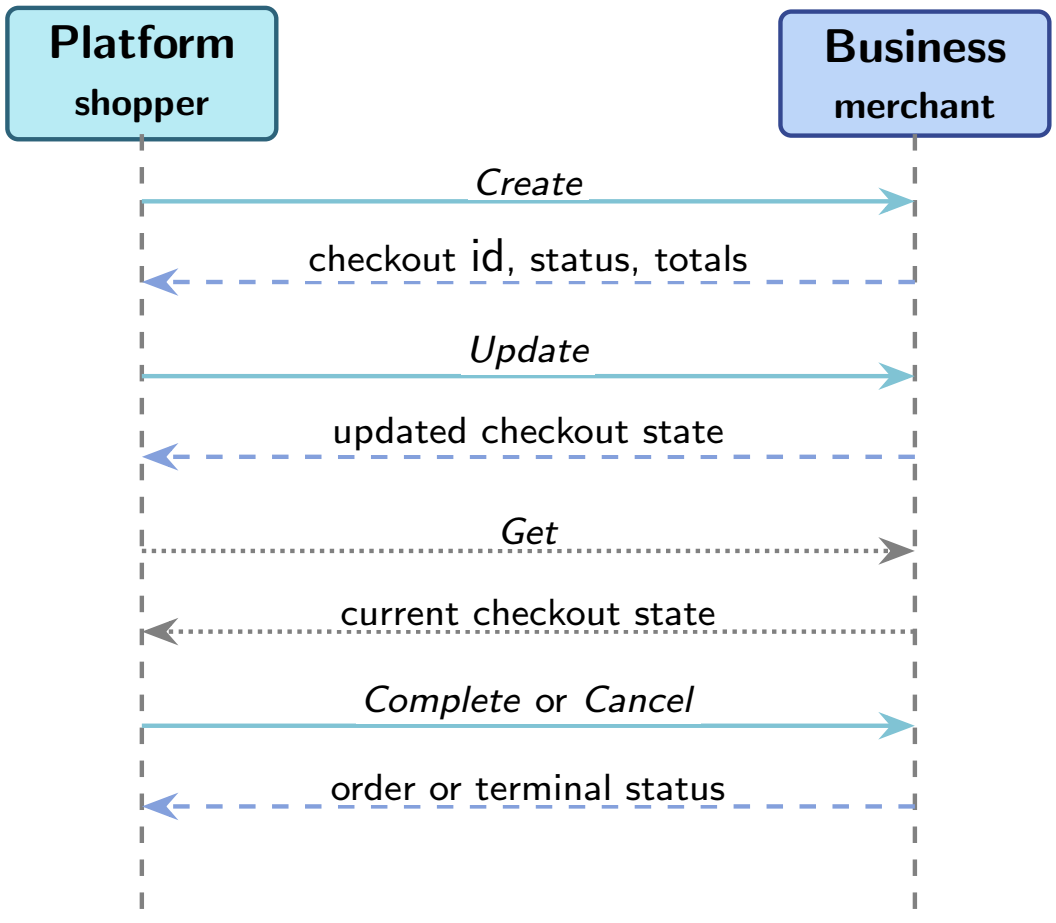


A2A: Infrastructure for the AI Age?

- Facilitates task delegation
- Orchestration mindset
- Not suited to multiprincipal settings
- Most interactions have nothing to do with delegation
- Repeats past mistakes (of FIPA ACL and KQML)

Universal Commerce Protocol (UCP)

Message schemas
Sequence of request-response exchanges
No formal representation of meaning
Inflexibe



Operations

Create
Start a checkout; Business generates the session id

Update
Supply or overwrite checkout data

Get
Recover the current session state

Complete
Pay and create the order

Cancel
Abandon the checkout

Interaction-Oriented Programming (IOP)

Multiagent system

Agents + Interaction protocol

Method

- Model a multiagent system in terms of interactions
  - Meaning via norms to compute accountability
  - Operational protocols to determine information exchange
- Verify these models
- Use the models to realize agents

High-level abstractions: Natural language is imprecise

- Reflect stakeholder intuitions
- Let programmers (or LLMs) focus on the reasoning

Information Protocols in BSPL

Declarative, no control flow, no message ordering

Protocol specifies the computation of a decentralized information object (a tuple) via messages

- Messages contain information and convey meaning
- Keys identify enactments (business transactions) and support correlation and integrity
- Causality adornments determine when a role may send a message

Think of Message Meaning

Toward specifying a protocol for conducting ebusiness

```
Ebusiness {
  roles Buyer, Seller, Bank
  parameters ID key, Item, Price, Status

  Seller -> Buyer: offer[Id key, Item, Price]

  Buyer  -> Seller: accept[Id key, Item, Price, Decision]

  Buyer  -> Bank: instruct[Id key, Price, Details]

  Bank   -> Seller: transfer[Id key, Price, Details, Payment]

  Seller -> Buyer: shipment[Id key, Item, Price, Status]

  Seller -> Bank:  refund[Id key, Item, Payment, Amount, Status]
}
```

Think of Causality

accept, instruct, and shipment (anytime) after offer; transfer after instruct; refund after transfer and mutually exclusive with shipment

```
Ebusiness {
  roles Buyer, Seller, Bank
  parameters out Id key, out Item, out Price, out Status

  Seller -> Buyer: offer[out Id key, out Item, out Price]

  Buyer  -> Seller: accept[in Id key, in Item, in Price, out
      Decision]

  Buyer  -> Bank: instruct[in Id key, in Price, out Details]

  Bank   -> Seller: transfer[in Id key, in Price, in Details,
      out Payment]

  Seller -> Buyer: shipment[in Id key, in Item, in Price, out
      Status]

  Seller -> Bank:  refund[in Id key, in Item, in Payment, out
      Amount, out Status]
}
```

Flexibility: Ebusiness has 658 distinct enactments

Output from BSPL protocol verifier

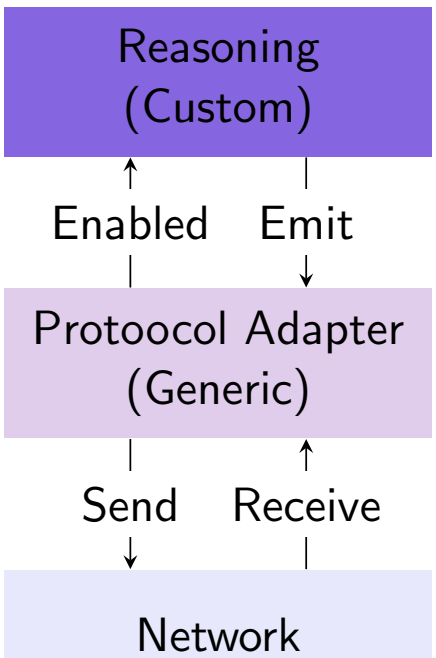
```
>bspl verify all_paths Ebusiness.bspl --verbose
(Seller!offer, Seller!shipment, Buyer?shipment, Buyer!
   accept, Buyer!instruct, Bank?instruct, Buyer?offer,
   Bank!transfer, Seller?accept, Seller?transfer)
(Seller!offer, Buyer?offer, Buyer!instruct, Buyer!
   accept, Bank?instruct, Bank!transfer, Seller?accept,
   Seller!shipment, Buyer?shipment, Seller?transfer)
...
```

Agent Programming Model Based on Protocols (with Samuel H. Christie)

Agent's reasoning selects an information-enabled messages, fleshes it out, and emits it

offer(1, fig, £5)
Seller's Local State

offer(Id, Item, Price) shipment(1, fig, £5, Status)
Seller's Enabled Messages




```
@adapter.schedule(* 17 * * *)
def decision_shipment(enabled):
    es = enabled["shipment"]
    for s in es
        s["Status"] = Nextday
```

Flexible Declarative Protocols in Langshaw

Supports agent reasoning by emphasizing meaning, hiding coordination

Enactable over a shared memory or a decentralized infrastructure

```
1: Ebusiness  // Name of the protocol
2:  who  Buyer, Seller, Bank // Roles
3:  what  ID key, transfer, shipment or refund //Completion
4:  do  // Action schemas to change the social state
5:   Seller: offer(Id key, Item, Price)
6:   Buyer: accept(Id key, Item, Price)
7:   Buyer: instruct(Id key, Price, Details)
8:   Bank: transfer(Id key, Price, Details, Payment)
9:   Seller: shipment(Id key, Item, Price)
10:  Seller: refund(Id key, Item, Payment, Amount)
11:  sayso  // Who has social authority over information
12:  Buyer: Item, Price, Details
13:  Bank: Payment
14:  Seller: Amount
15:  nono // What action pairs are incompatible
16:    shipment refund
```

Specifying Normative Meaning Explicitly

```
1: commitment OfferCom Seller to Buyer   //if transfer, then
       shipment
2:    create offer
3:    detach transfer[, created OfferCom + 5]
4:           where "Payment>=Price"
5:    discharge shipment [, detached OfferCom + 5]

7: commitment AcceptCom ...//if shipment, then transfer
8: commitment RefundCom ...//if violated OfferCom, then refund
9: commitment TransferCom ...//if instructed, then transfer
```

Programming Model

Upon detachment of RefundCom, Seller sends Refund

```
+ev_now_detached_RefundCom(Seller, Buyer, Id, Item, Price,
    Bank, Payment, Timestamp)
  <- !send_refund(Id, Item, Payment, Bank).
```

Seller sends shipment if OfferCom is nearing violation

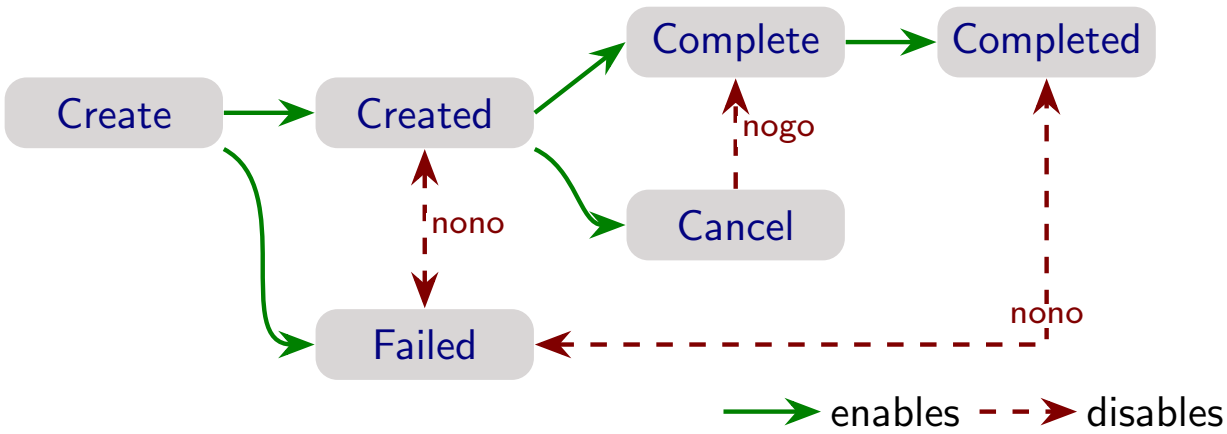
```
+!handle_form([shipment(Id, Item, Price, out)[receiver(
    Buyer)]|_])
  : in_stock(Item) & violated_OfferCom(Id,..., T) &
    system_time(Now) & T <= Now + 10
 <- !send_shipment(Id, Item, Price, Buyer).
```

Simplified UCP in Langshaw

```
1: SimpleUCP
2: who Platform, Business
3: what cid key, Completed or Failed
4: do Platform: Create(cid, line_items, currency, buyer,
       payment_pref, discount_codes, fulfillment_pref)
5: do Business: Created(cid, Create, id, status, totals,
       payment, discounts, fulfillment)
6: do Business: Failed(cid, reason)
7: do Platform: Complete(cid, Created, id, payment_data,
       risk_signals)
8: do Platform: Cancel(cid, Created, id)
9: do Business: Completed(cid, Complete, order)
10: sayso Platform: line_items, currency, buyer, discount_codes,
       fulfillment_pref, payment_pref, payment_data,
       risk_signals
11: sayso Business: id, status, totals, payment, discounts,
       fulfillment, order, reason
12: nono Completed Failed
13: nono Created Failed
14: nogo Cancel -/> Complete
```

SimpleUCP Highlights

- cid gives every enactment an identity before HTTP assigns an id
- **sayso** records authority
- **nono** and **nogo** make conflicts explicit



Parismony: the resulting PLATFORM and BUSINESS agents are order of magnitude smaller than Google's reference implementations

Multiagent System

Protocol captures interaction constraints

- Supports interoperation and loose coupling
- Lays out the possible interactions

Agent represents a principal and captures their internal reasoning

- Helps automate interactions
- Chooses from the possible interactions

LLM agents that select and participate in a protocol

- *How can we exploit LLMs to create appropriately flexible agents without programming them?*

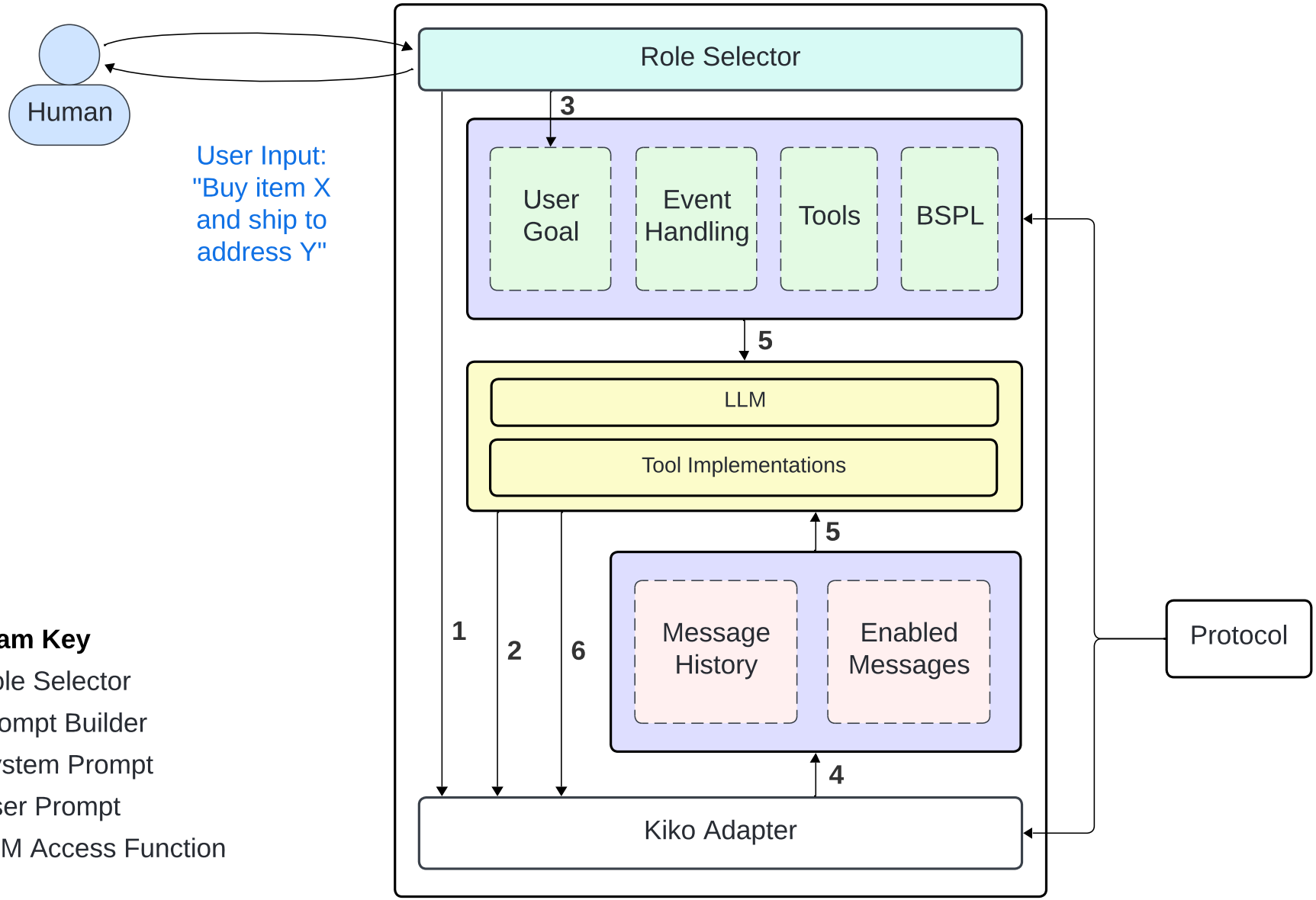
LLM Agents Promise Flexibility But Lack Rigor

```
## Pure LLM agent
response = client.chat.completions.create(
  model="gpt-4",
  messages=[
    {"role": "system", "content": "You are a helpful assistant and you need to achieve " + [User Goal]},
    {"role": "user", "content": [Decision Context] + " Decide what the next step is."}
  ]
)

print(response.choices[0].message.content)

## Example output
## (Not a message defined in the protocol)
{
  "message": "buy",
  "item": "pen",
  "price": 20,
}
```

Ahoy Architecture: LLM Agent Designed for Interaction



Configuring an Ahoy Agent

User goal

```
Describe the scenario or goal you'd like ahoy to participate in.
(Enter multiple lines. Type 'END' on a new line to finish)

I need to buy a pen for less than $20.
Delivery location: Raleigh, NC 27606
END
```

Available protocols (from a repository)

```
================================================================
Available Protocols
================================================================

1. Purchase: [Buyer, Seller, Shipper]
2. CreditPurchase: [CreditBuyer, CreditSeller, CreditShipper]
3. Logistics: [Merchant, Wrapper, Labeler, Packer]
4. FlexiblePurchase: [FlexibleCustomer, FlexibleMerchant]
5. NetBill: [Customer, Merchant]

Select a protocol (number or name):
```

Ahoy Agent Chooses a Message and Binds Its Parameters

```
================================================================================
2026-04-27 20:03:53,667 - DEBUG - RAW LLM RESPONSE
2026-04-27 20:03:53,667 - DEBUG - ================================================================================
2026-04-27 20:03:53,667 - DEBUG - Looking at my goal and the current state:

**Goal Analysis:**
- Need to buy a pen for less than $20
- Delivery location: Raleigh, NC 27606

**Current Situation:**
- I've already requested a quote for a pen
- The seller quoted $3 for a pen
- The price ($3) is well under my $20 budget ✓
- I need to accept this offer and proceed with delivery

**Decision:**
Option 1 (Purchase/accept) is the appropriate next step. I should accept the quote with my delivery address.

```json
{
  "choice": 1,
  "params": {
    "address": "Raleigh, NC 27606",
    "resp": "Accepted - proceeding with purchase"
  },
  "tool_requests": []
}
```
```

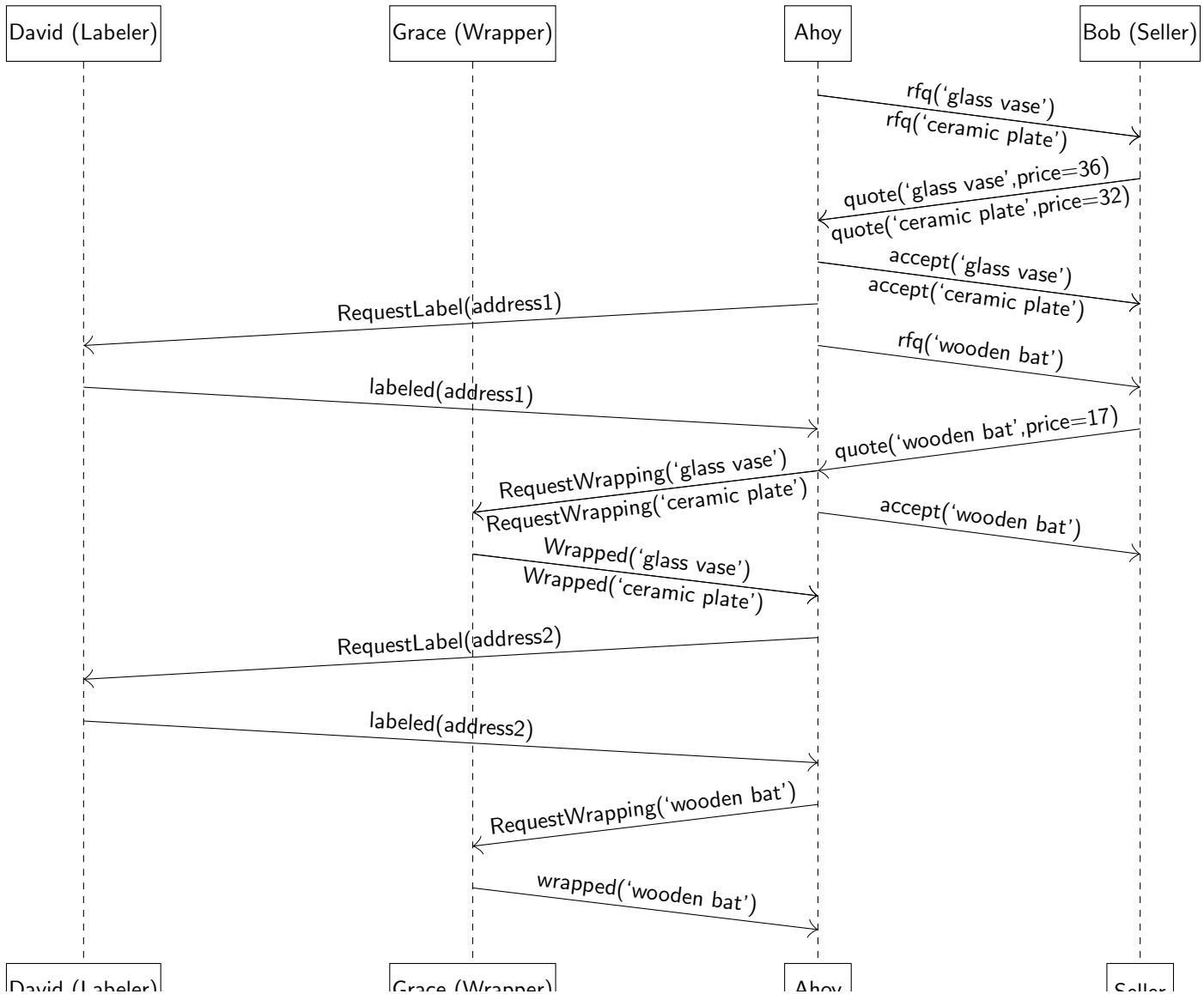
Programming Not Needed

Task: "Buy a pen for $20 an deliver it to address . . .".

```
rfq(ID=[ID],item='pen'){protocol='Purchase'}
accept(ID=[ID],item='pen',price=15,address=[user_address
    ],resp='Accepted. Please proceed with shipment to
    Raleigh, NC 27606.'){protocol='Purchase'}
completed(ID=[ID],item='pen',price=15,satisfaction='
    Transaction completed successfully. Pen received in
    good condition at [user_address].'){protocol='
    Purchase'}
```

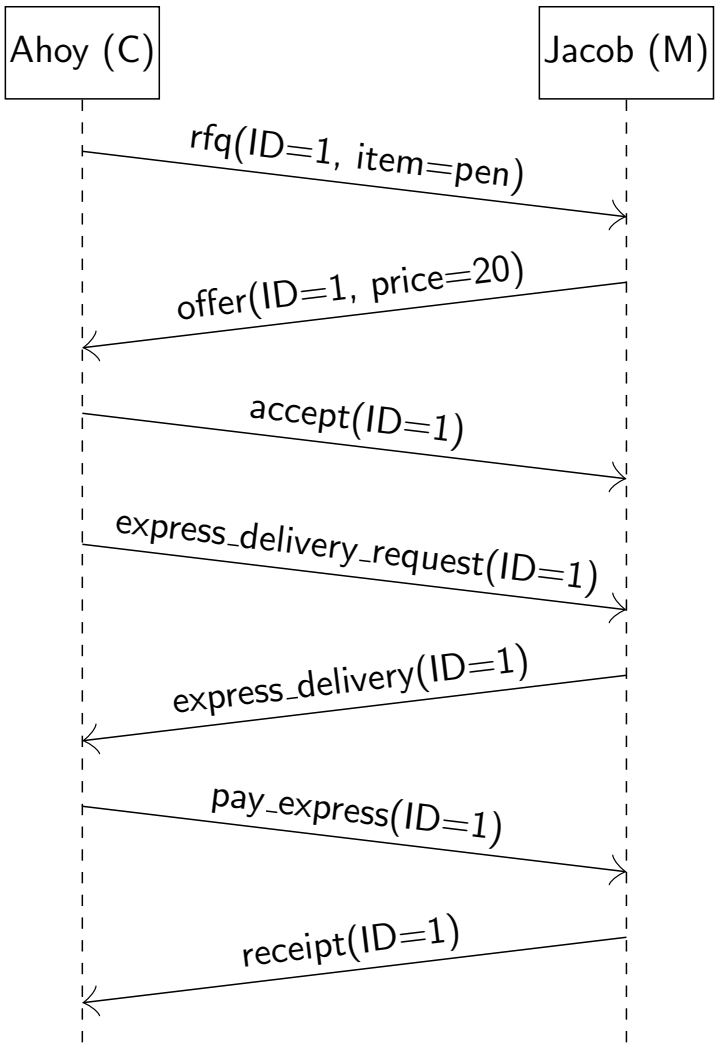
Concurrent Participation in Two Protocols

"Buy a glass vase and a ceramic plate and ship to address1, and buy a bat and ship to address2. The total budget is $200."

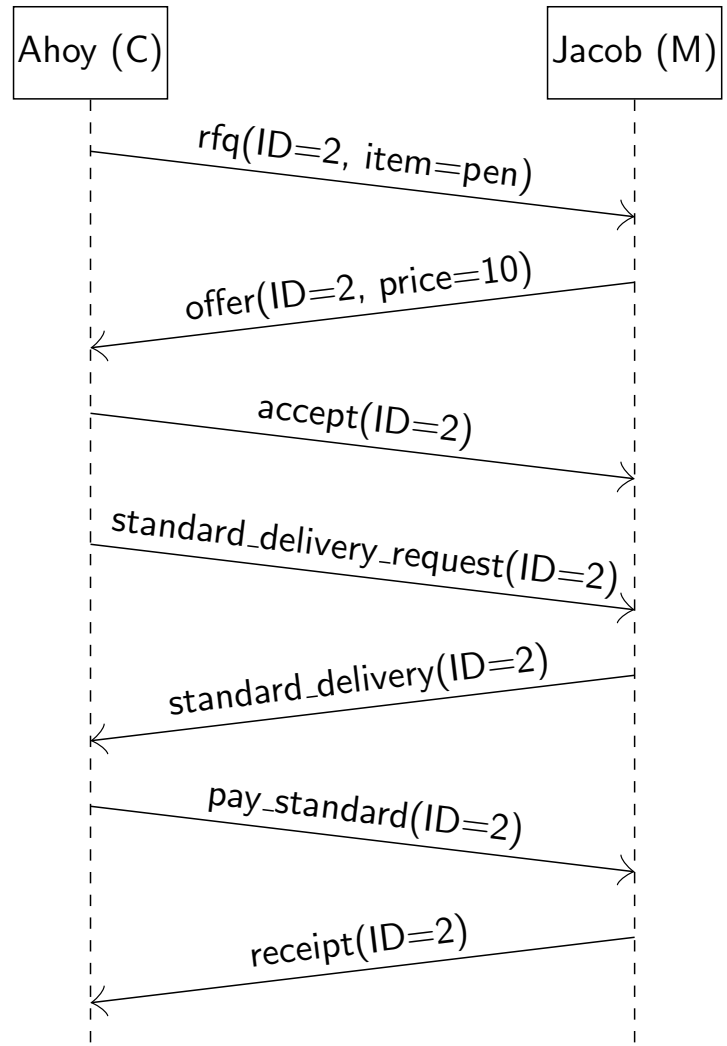


Flexibly Enacting Branching Protocols: Alternative Paths

I want a pen delivered urgently

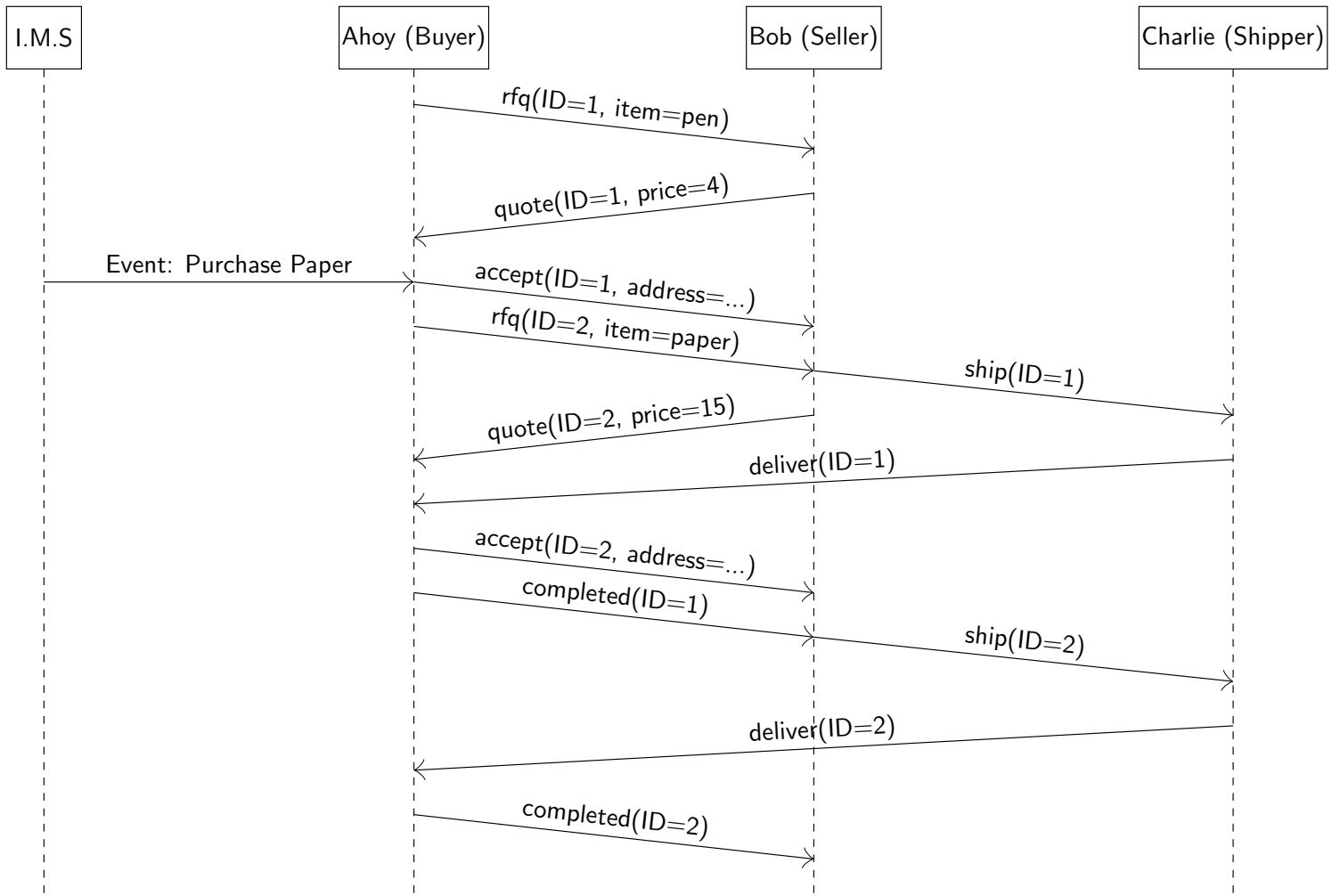


I want a pen delivered cheaply



External Event Handling

"Buy a pen and deliver to address."



Evaluation: Ahoy Agents

- Respect protocol semantics
- Select actions intelligently given user goals
- Can interleave multiple protocol enactments
- Handle exogenous events

Problem	Solution
Poor understanding of interaction: mapped goal to incorrect protocol	Ongoing work to support norms
Hallucination: hallucinated ID values	Generated and passed IDs
Insufficient commonsense knowledge: Bought pen for $10k	Budget given in goal
Poor knowledge of business practices: Kept checking with seller repeatedly	Explicitly instructed to not spam seller

Norms and Declarative Protocols $\mapsto$ Multiagentic AI

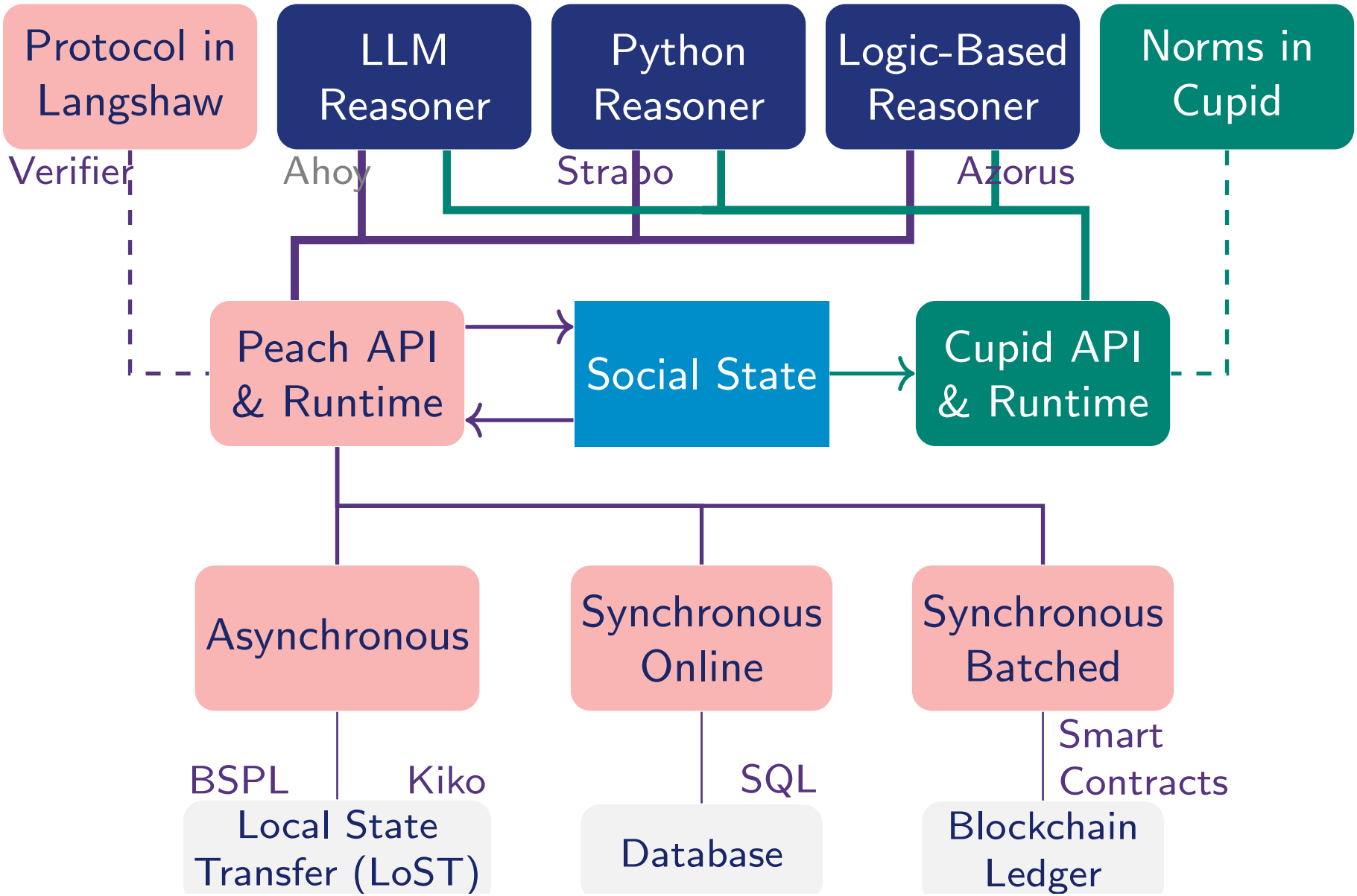
Conventional Agentic AI	Interaction Orientation
Orchestration	Interactions
Natural language: ambiguous	Formal, declarative, flexible
Over-constrained communication	Flexible communication
Soft guardrails	Hard guardrails
LLM agents	Theory based (LLM . . .) agents

Engineering is a science of the artificial (Simon)

Models capture *requirements* of the engineered artifact

Interaction-Oriented Programming Ripe for Standardization

Lots of running code, just need rough consensus



FIPA ACL & KQML

Of historical importance only

- Inspired from Austin's speech acts
- Handful of act types, e.g., inform, promise, achieve, ...
- KQML informal, intended for closed settings
- FIPA ACL semantics in terms of mental states
  - To promise something means speaker intends to do that thing
  - To assert something means speaker believes it
- Adopted in agent programming languages and frameworks, e.g., Jason and JADE

Limitations

- Types of communicative acts in practice are virtually unlimited
- Impossible to determine compliance in open systems

Requirements for Multiagent Interaction Protocols

Munindar P. Singh[2] Amit K. Chopra[1] Samuel H. Christie V[2]
(with contributions from Matteo Baldoni[3] and Omkar Joshi[2])

[1]Lancaster University

[2]North Carolina State University

[3]University of Torino

Papers

Code

W3C TPAC 2026